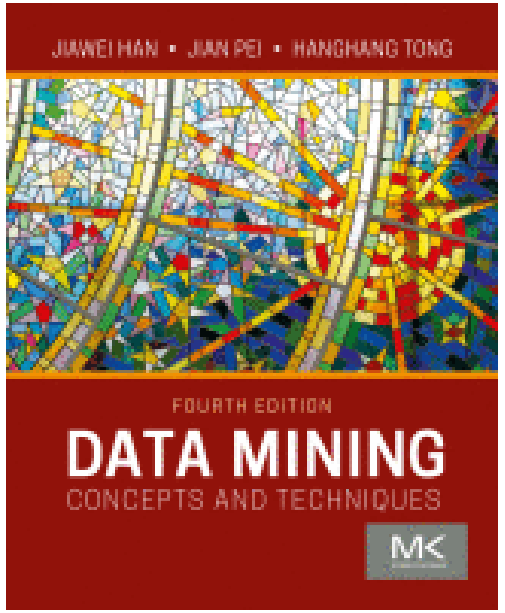
Deep Learning for Text Data

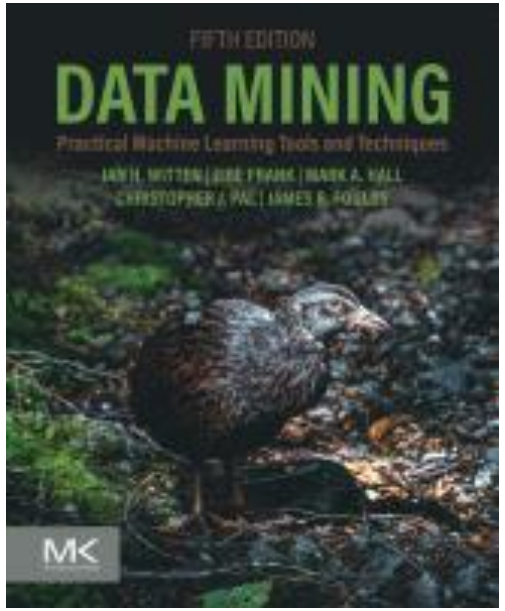
IF5141 dan IF4041 - Penambangan Data

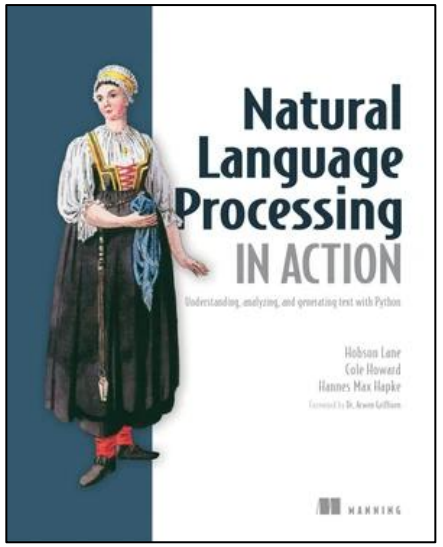
©2026 - Tim Pengajar IF4041 dan IF5141 Semester I TA 2026/2027

References

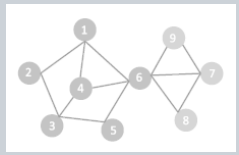
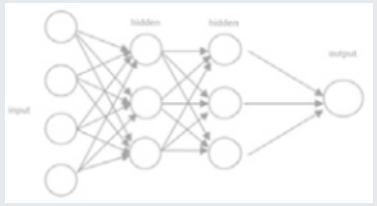
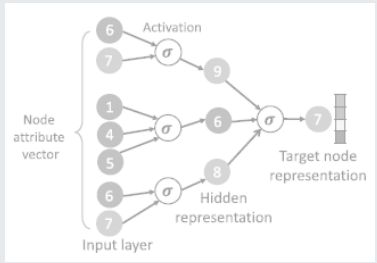
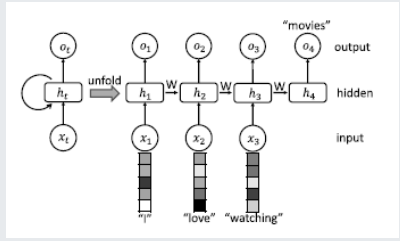
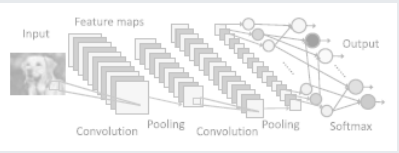
- Han, J., Pei, J., & Tong, H. (2024). *Data mining: Concepts and Techniques*. Morgan Kaufmann. https://doi.org/10.1016/C2013-0-18660-6
- Witten, I. H., Hall, M. A., Foulds, J. R., Frank, E., & Pal, C. J. (2025). *Data mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann. https://doi.org/10.1016/c2022-0-02092-4
- Lane, H., & Dyshel, M. (2025). *Natural Language Processing in Action, Second Edition*. Manning.







Overview of deep learning architecture

Data Type	*Multidimensional data* (i.e., tabular)	*Relational* data (i.e., graph)	*Sequence* data (i.e., text)	*Grid-like* data (i.e., image)
			x = "I love watching movies."	
Deep Learning (DL) Architecture	Feed-forward neural networks 	Graph neural networks 	Recurrent neural networks 	Convolutional neural networks 

Another kind of deep neural network (which is designed for other types of input data) is a graph neural network (GNN) for *relational* (i.e., graph) data.

Text Data

Text Data

- Data in the world can be organized:
  - in a highly **structured** form, such as that in a typical relational database,
  - in a **semi-structured** data format, which is a blend between structured and unstructured data, such as XML, JSON, and HTML files,
  - in an **unstructured** format, such as:
    - text data from various written sources such as books, emails, reviews, articles, websites, news, product reviews, or social media or
    - in rich media formats like video, image, and audio data.
- Text (data) mining, also called text analytics, is the process of deriving high-quality information, such as structures, patterns, and summaries, from text.

Text Data Mining

Text Classification

- *Text classification* (also known as *text categorization*), is to analyze open-ended texts (e.g., documents, messages, or webpages) and assign them to a set of predefined topics or categories.
- A supervised approach takes a good amount of training data (i.e., annotated by experts or tagged with human feedback) to construct a quality classification model.
- It has broad applications such as sentiment analysis, topic labeling, spam detection, and intent detection.

Text Clustering

- *Text clustering* is to group a set of unlabeled texts (e.g., documents, sentences, words) in such a way that texts in the same cluster are more similar to each other than to those in other clusters.
- Text clustering algorithms process texts and determine if natural clusters (groups) exist in the data based on some similarity measures.
- Text clustering can benefit many text analysis tasks, such as document retrieval (adding similar documents to improve recall) and social media analysis.

Information Extraction (IE)

- Information extraction is the task of automatically extracting structured information from unstructured and/or semi-structured documents.
- One of the most typical **subtasks** in IE is *named entity recognition* (NER), which recognizes known entity names (for people and organizations), locations, temporal expressions, and certain types of numerical expressions.
- Although human-annotated documents have typically been used for training NER models, recent studies have been using Wikipedia or other existing knowledge bases, pretrained language models, and text embedding methods.

Text Summarization

- Automatic text summarization is the process of creating a subset of text that represents the most important or relevant information from a single piece or multiple pieces of text (typically, document).
- Based on the text to be summarized, text summarization can be categorized as *single-document* or *multi-document summarization*.
- There are two general approaches to text summarization:
  - *Extractive summarization* extracts content (e.g., key sentences or key phrases) from the original data, without modification.
  - *Abstractive summarization* builds an internal semantic representation of the original content and then uses it to create a summary that is closer to what a human might express.

Preparing Text Data for DL

Tokens and Tokenization

- Tokenization is a particular kind of document segmentation.
  - Documents can be segmented into paragraphs, paragraphs into sentences, sentences into phrases, and phrases into tokens (usually words and punctuation).
- A tokenizer breaks unstructured data, natural language text, into chunks of information, which can be counted as discrete elements.
  - The simplest way to tokenize a sentence is to use whitespace in a string as the "delimiter" of words.

```
>>> text = ("Trust me, though, the words were on their way, and when "
...         "they arrived, Liesel would hold them in her hands like "
...         "the clouds, and she would wring them out, like the rain.")
>>> tokens = text.split()
>>> tokens[:8]
['Trust', 'me,', 'though,', 'the', 'words', 'were', 'on', 'their']
```

str.split() is your quick-and-dirty tokenizer.

Text Representation

- A fundamentally important issue for text mining is how to represent text primitives (e.g., words, phrases, sentences, and documents) and how to learn effective representation from unstructured text.
- We can use:
  - Symbol-based representation
  - Distributed representation

Symbol-based representation

- Early studies use symbol-based representation, such as:
  - **"One-hot encoding" vectors:** assigning 1 at the word appearing position and 0 at all other positions.

Table 11.1 One-hot encoding representations for words. Each column contains a representation for one of the words in the document, with a single one at the entry corresponding to the index of that word in the dictionary and zeros elsewhere

Index	Document Word	The	quick	brown	fox	jumps	over	the	sly	lazy	dog
1	The	1	0	0	0	0	0	1	0	0	0
2	Over	0	0	0	0	0	1	0	0	0	0
3	Quick	0	1	0	0	0	0	0	0	0	0
⋮											
V		0	0	0	0	0	0	0	0	0	0

Credit: James Foulds

  - This representation does not contain any information about the meaning of the words or their relationships to each other.

Distributed representation

- Recent studies have been using distributed representation, such as:
  - **Word embedding:** each object (e.g., word) is represented (or encoded) with a low-dimensional real-valued dense vector

Table 11.2 Word embedding representations for words. Each column is a word embedding for the corresponding word in the document.

Dimension	The	quick	brown	fox	jumps	over	the	sly	lazy	dog
1	−0.71	0.83	0.99	0.62	1.24	−3.06	−0.71	1.37	−0.51	0.59
2	−0.2	−2.00	−3.98	−1.26	0.95	0.55	−0.2	0.01	−0.36	−1.50
3	0.11	0.65	0.90	0.33	0.41	0.29	0.11	0.84	0.72	0.4
⋮										
K	1.21	0.76	1.51	2.00	0.34	0.10	1.21	4.65	2.59	−1.22

Credit: James Foulds

Word Embedding

- Word embeddings use the similarity of vectors to encode similarity of meaning.
  - Each column of the table is a word embedding (a.k.a. a word vector) for the corresponding word.
  - The dimensionality K of the embeddings is a user-selected hyperparameter.
- There are various methods to generate such embeddings, such as neural networks.

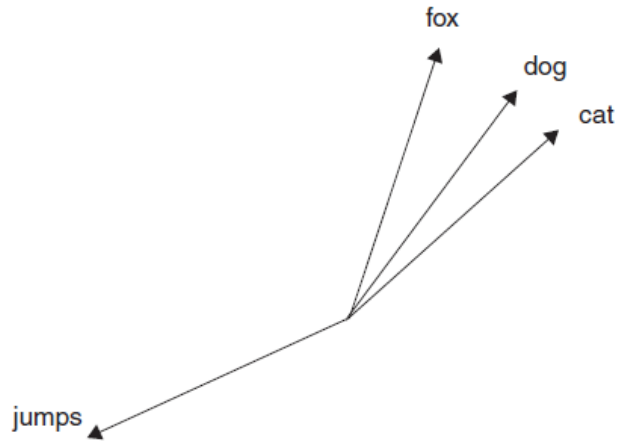


FIG. 11.3

Word embeddings encode semantic similarity relationships. Here, *dog* and *cat* are closely related words, so they have similar vector-space embeddings. *Dog* and *fox* are somewhat similar and have somewhat similar vectors, while *jumps* is very different to the other words and has a very different vector.

Credit: James Foulds

Table 11.2 Word embedding representations for words. Each column is a word embedding for the corresponding word in the document.

Dimension	The	quick	brown	fox	jumps	over	the	sly	lazy	dog
1	−0.71	0.83	0.99	0.62	1.24	−3.06	−0.71	1.37	−0.51	0.59
2	−0.2	−2.00	−3.98	−1.26	0.95	0.55	−0.2	0.01	−0.36	−1.50
3	0.11	0.65	0.90	0.33	0.41	0.29	0.11	0.84	0.72	0.4
⋮										
K	1.21	0.76	1.51	2.00	0.34	0.10	1.21	4.65	2.59	−1.22

Credit: James Foulds

Word Embedding: Word2Vec

- Word2vec is a word embedding method developed around 2013.
- Word2vec is a two-layer neural network that processes text by taking a large corpus of text as its input and producing a vector space
  - The vector typically has several hundred dimensions.
  - Each unique word in the corpus being assigned a corresponding vector in the space.
- Given enough data, Word2vec can make accurate guesses about a **word's meaning based on past appearances**.

Pretrained language models

- Pretrained language models, such as BERT (around 2018) use larger corpora, more parameters, and more computing resources **than word embedding methods** to pretrain large and effective models on large corpora.
- They use multilayer neural networks to calculate dynamic representations for the words based on their context.
  - A word should have different meanings in different contexts. For example, the word *bank* in the name *World Bank* and in the expression *riverbank*.
  - *Contextualized word embeddings* can be updated or refined.

DL Architecture

RNN architectures

Sequence Data (Text)

- The input is represented as a sequence of values $(x^1, x^2, \dots, x^T)$.
  - The sequence could be a sentence.
- T is the length of the sequence.
  - T is the total length (number of words) of the sentence.
- x^t is the value at a particular position or time stamp t.
  - x^t is the tth word of the input sentence.
  - $t = 1, \dots, T$
- The key is to model the *sequential dependence*.
  - Different words in the same sentence are likely to be correlated with each other.

FFNN vs RNN

- How to use previous information to affect later ones? *(sequential dependence)*
- A recurrent neural network (RNN) processes your text one token at a time.
  - To keep track of the tokens it has already read, it records a hidden vector (h) (also called a state) that can be passed along to its future self.

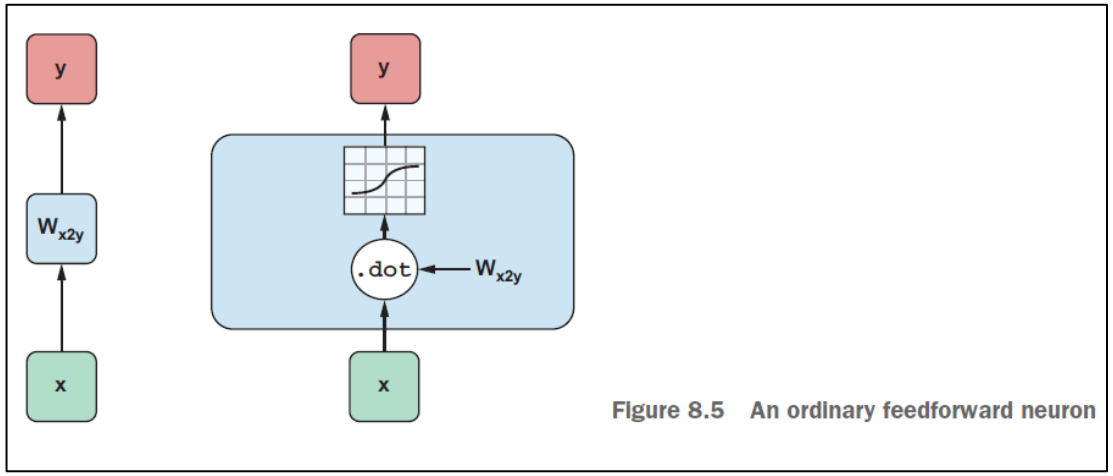


Figure 8.5 An ordinary feedforward neuron

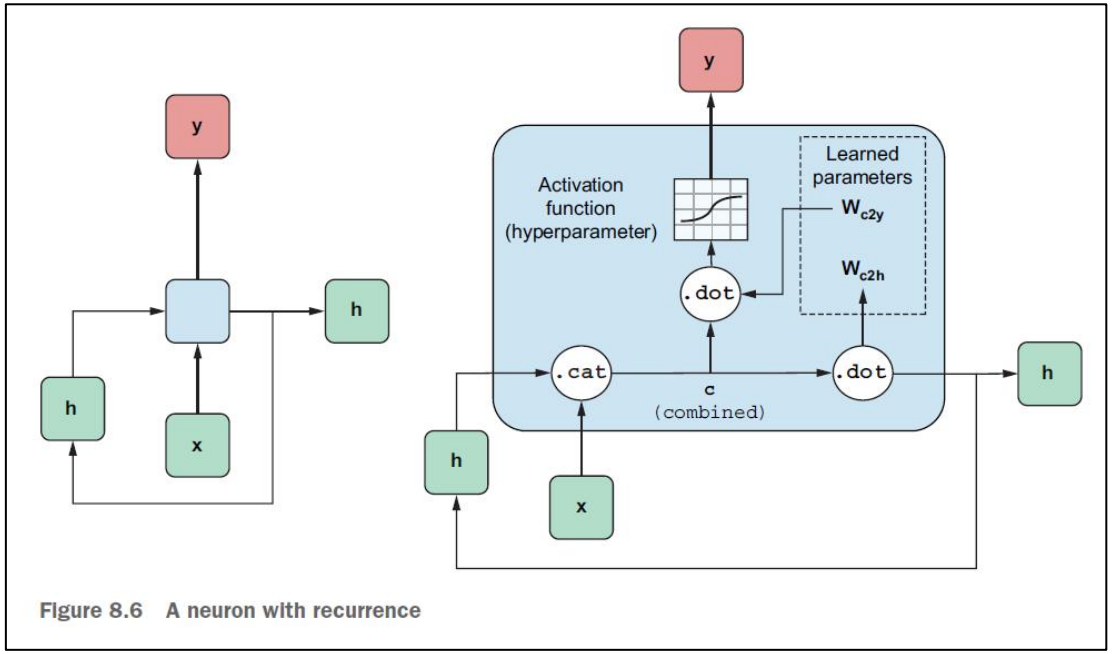


Figure 8.6 A neuron with recurrence

RNN Example

- Given a (partial) sentence "Illinois is the land of".
- We want to predict the next possible word (e.g., "Lincoln" or "Corn").
- We can use an RNN model:

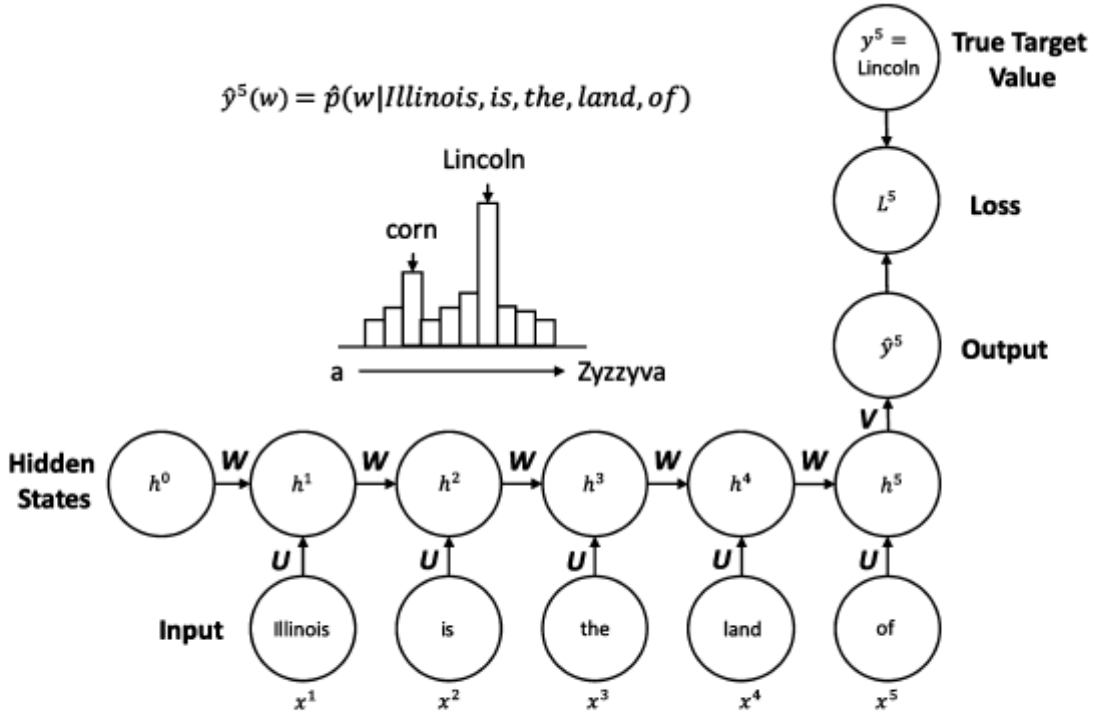
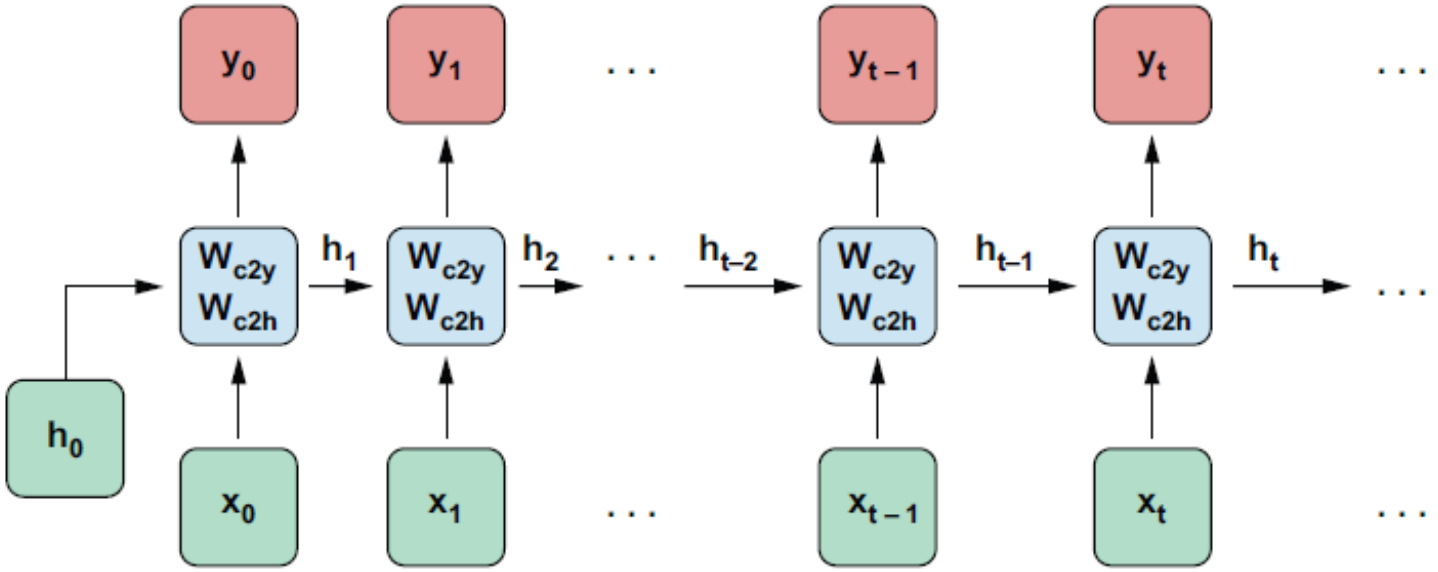


FIGURE 10.29

An example of RNNs, consisting of five input units, five hidden units, and one output unit.

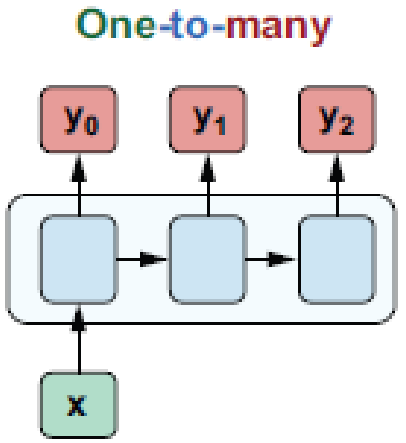
Recurrent Neural Network (RNN)

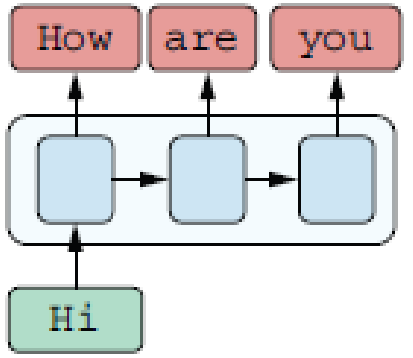
- An RNN passes the hidden state (h) along to the next future-self neuron.



Inputs and Outputs of RNNs (1)

Type	Description	Applications
One to many	One input tensor used to generate a sequence of output tensors	Generate chat messages, answer questions, describe images

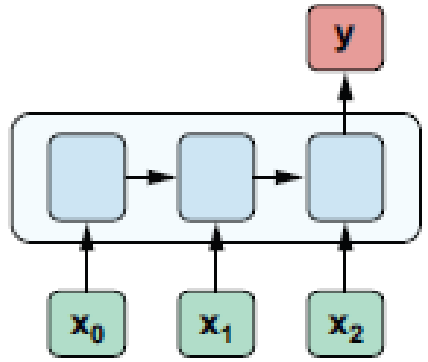


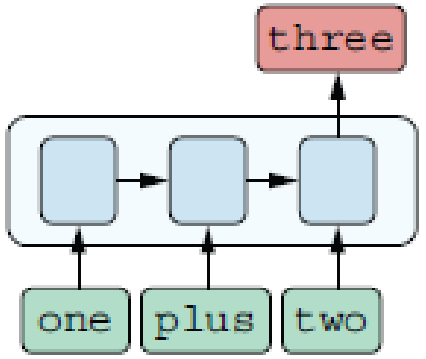


Inputs and Outputs of RNNs (2)

Type	Description	Applications
One to many	One input tensor used to generate a sequence of output tensors	Generate chat messages, answer questions, describe images
Many to one	A sequence of input tensors gathered up into a single output tensor	Classify or tag text according to its language, intent, or other characteristics

Many-to-one

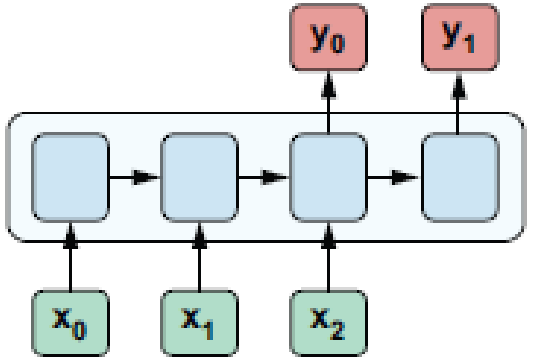


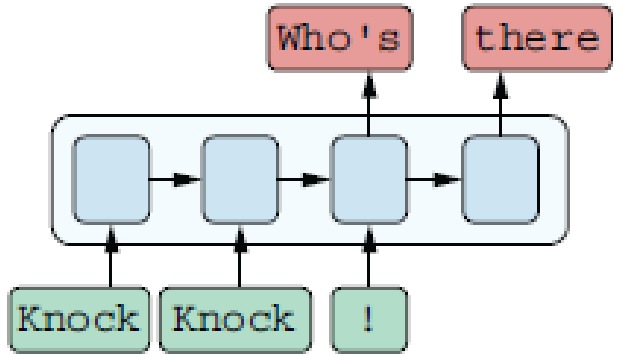


Inputs and Outputs of RNNs (3)

Type	Description	Applications
One to many	One input tensor used to generate a sequence of output tensors	Generate chat messages, answer questions, describe images
Many to one	A sequence of input tensors gathered up into a single output tensor	Classify or tag text according to its language, intent, or other characteristics
Many to many	A sequence of input tensors used to generate a sequence of output tensors	Translate, tag, or anonymize the tokens within a sequence of tokens, answer questions, participate in a conversation

Many-to-many





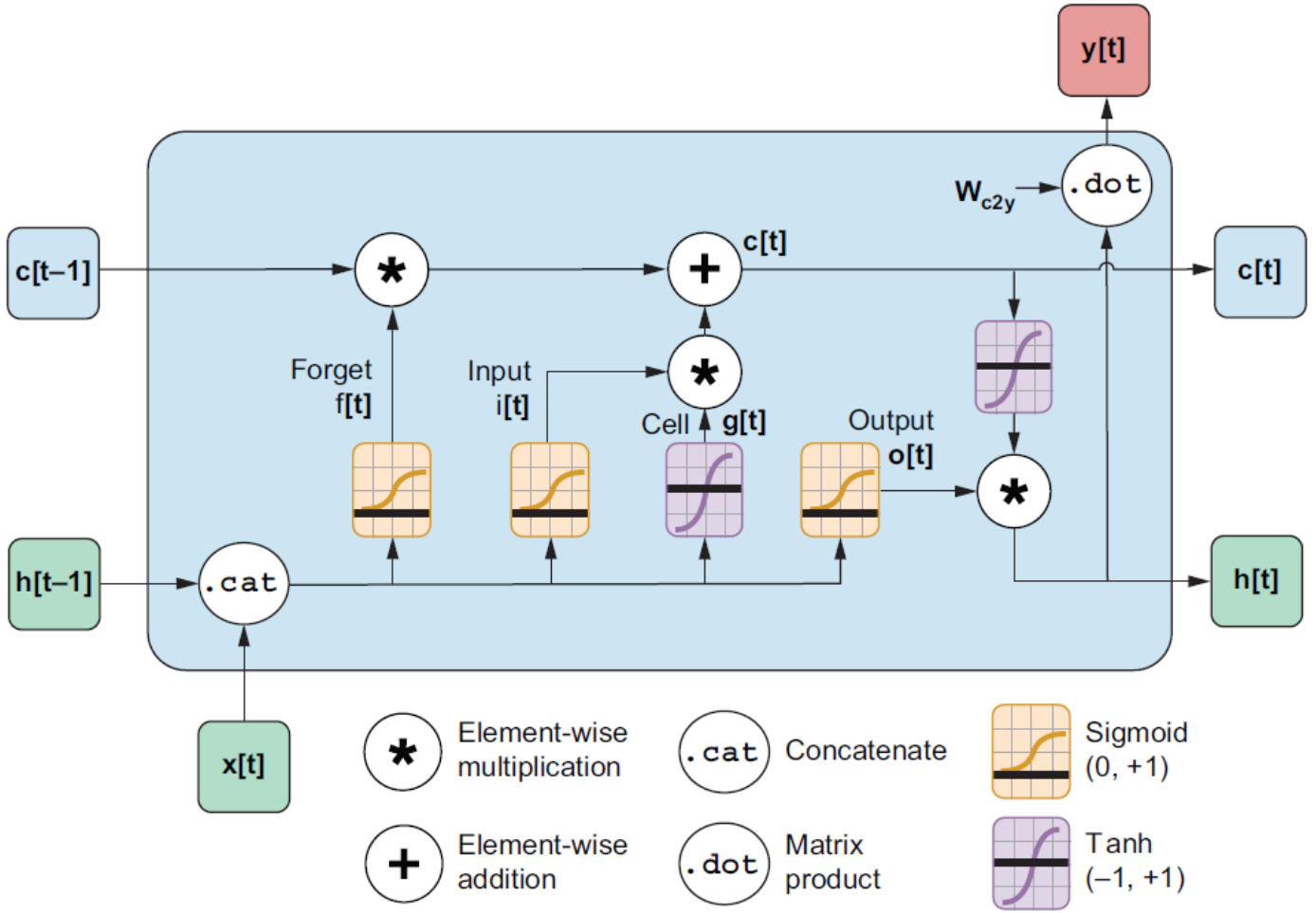
Basic RNN vs Gated RNN

- Ordinary RNNs have a short attention span that limits their ability to understand longer texts.
  - The influence of the first token in a string fades over time, as your machine reads more and more of the text.
- That's the problem that *gated recurrent units* (GRUs) and *long short-term memory* (LSTM) neural networks aim to fix.
  - Those model equipped with gate functions. Thus, they are called **gated RNNs.**

Lane (2025); Han (2024)

Gated RNN: Long Short-term Memory (LSTM)

- Besides the hidden state, an LSTM has a separate cell state to act as its long-term memory.
- LSTMs have 3 gates:
  - **Forget Gate**: controls how much information of the old cell state is forgotten.
  - **Input Gate**: controls how much new information from the temporary cell state is used to update the cell state vector.
  - **Output Gate**: controls how much information of the new cell state is used to generate the new hidden state.



Gated RNN: Gated Recurrent Units (GRU)

- GRUs merge the cell state and hidden state.
- GRUs have 2 gates:
  - **Reset Gate**: Decides how much past information should be forgotten.
  - **Update Gate**: Decides how much past information should be kept.

